System Calls

Lecture 3 — Mechanics, Parameter Passing, and Types

Dr. Auqib Hamid Lone
Assistant Professor in Computer Science & Engineering

PCC CS-502: Operating Systems

Department of Computer Science & Engineering
Government College of Engineering & Technology (GCET), Ganderbal

October 29, 2026

Welcome: How Many Times Does cp Enter the Kernel?

The big question

You run `cp notes.txt backup.txt`. The command opens one file, creates another, copies every byte, and closes both. How many times does the program cross from user mode into the kernel — and what does it say each time?

- cp is ordinary user code. It cannot touch the disk, the page tables, or another process's memory.
- Every privileged operation must be requested through a **system call**.
- The request has a name, a number, arguments, and a return value.

Running thread for today

We follow cp through its system calls and open up the mechanism that carries each request across the boundary.

Roadmap: Four Questions About the Doorway

1. **What is a system call?** — the kernel's programming interface.
2. **How does control cross the boundary?** — the trap and the system-call table.
3. **How do arguments travel?** — registers, memory blocks, and the stack.
4. **What kinds of calls exist?** — the six standard categories.

The mechanism is hardware; the interface is software; the taxonomy is the vocabulary you will use for the rest of the course.

Programmers rarely invoke a system call directly. They call a library function — and the library eventually executes the trap.

Two Views of the Same Service

Programmer's view (API)	**Kernel's view (system call)**
`fopen`, `fread`, `fwrite`, `fclose`	`open`, `read`, `write`, `close`
Standard C library; buffered; portable	Kernel entry points; unbuffered; architecture-specific
Returns a `FILE *` or a count	Returns a file descriptor or -1 with `errno`

- The API is a *convenience layer*; the system call is the *actual request* to the kernel.
- One library call may issue zero, one, or many system calls (buffering changes the count).

Silberschatz et al., Ch. 2 §2.3, pp. 55–58.

What Exactly Is a System Call?

Definition

A **system call** is the programmatic request from a user program to a kernel service. It is the boundary of the operating system's programming interface.

- It looks like a function call in C, but it is *not* an ordinary function call.
- It changes the privilege level: the CPU enters kernel mode, runs kernel code, and returns to user mode.
- The kernel executes it on behalf of the calling process, with the kernel's privileges — after validating the request.

The same word is used for the API function, the trap instruction, and the kernel routine; context tells you which layer is meant.

Why Not Link the Kernel Into Every Program?

- **Protection:** user code would run with kernel privileges and could corrupt the machine.
- **Isolation:** one buggy program could overwrite another's memory or the kernel's data structures.
- **Portability:** the same API can sit on different kernels and instruction sets.
- **Evolution:** the kernel can change its internals without recompiling every application.

The trade-off

Every system call costs a mode switch. The design goal is to keep the boundary narrow and the calls efficient.

Socratic Check 1

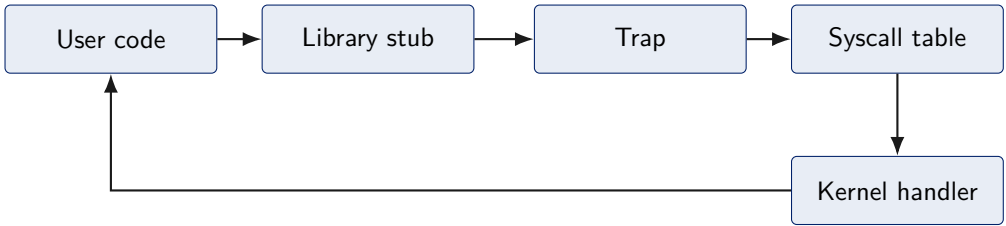
Question

`printf` is not a system call. What does it eventually call, and why does the C library buffer the output first?

- `printf` formats into a user-space buffer.
- When the buffer fills or is flushed, the library calls `write`, which traps into the kernel.
- Buffering reduces the number of expensive mode switches.

The trap instruction is the hinge: one instruction moves the CPU from user mode to kernel mode and hands it a number.

The Trap Path



- The stub places the **system-call number** and arguments in registers.
- The trap saves the user PC and flags, switches to kernel mode, and jumps to the kernel's trap entry point.
- The kernel indexes the **system-call table** by number and calls the handler.

Worked Trace: `write(fd, buf, n)`

1. `fd`, `buf`, and `n` are placed in the argument registers.
2. The system-call number for `write` is placed in a designated register.
3. The stub executes `syscall` (x86-64) or `int 0x80` (legacy x86).
4. Hardware switches to kernel mode, saves the return address, and enters the trap handler.
5. The kernel validates `fd`, checks that `buf` lies in user memory, and performs the write.
6. The return value is placed in a register; the kernel restores user mode and returns.
7. On error, the library sets `errno` and returns -1.

Bovet & Cesati, Ch. 10, pp. 398–419; Silberschatz et al., Ch. 2 §2.3.

The System-Call Table

- A kernel array indexed by system-call number.
- Each entry is a function pointer to the handler.
- The number is the **ABI**: it must stay stable across kernel versions.
- Adding a call means adding a table entry and a handler — not changing the trap instruction.

Why a table?

The trap instruction carries only a number. The table turns that number into the right kernel routine without a long search.

On Linux x86-64, `write` is system call 1; the table lives in the kernel's trap entry code.

Return Values and `errno`

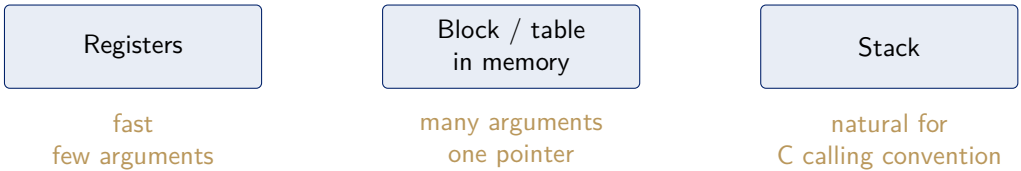
- The kernel returns a value in a register: a count, a file descriptor, or an error code.
- A negative return value signals an error; the library translates it into `-1` and sets `errno`.
- The caller must check the return value — the kernel cannot print a message for it.
- The kernel never trusts user pointers or lengths; it validates them before copying.

Example

`read` returns the number of bytes read, 0 at end of file, or -1 on error with `errno` set to `EFAULT`, `EBADF`, or similar.

A system call is a function call across a privilege boundary. The arguments must cross too — and the kernel must not trust any of them.

Three Ways to Pass Arguments



- Silberschatz describes all three; real ABIs choose one or a mix.
- The kernel must know where to find the arguments before it can validate them.

Method 1: Registers

- The caller places the first few arguments in designated registers.
- Fast: no memory access beyond the registers already saved on entry.
- Limited by the number of argument registers.

Linux x86-64 `read(fd, buf, count)`

`rax`	system-call number for `read`
`rdi`	`fd`
`rsi`	`buf` (user virtual address)
`rdx`	`count`

The kernel still checks that `buf` is a legal user address.

Method 2: Block or Table in Memory

- The caller stores the arguments in a memory block.
- A register holds the *address* of the block.
- The kernel copies the block into kernel space and validates every field.

A call with many arguments

`open(path, flags, mode)` can be passed as a table holding the pointer to `path`, `flags`, and `mode`; the kernel reads the table once.

The pointer itself is untrusted: the kernel must verify that the whole block lies in the caller's address space.

Method 3: Stack

- The caller pushes arguments onto the user stack.
- The kernel reads them from the saved user stack.
- Natural for languages and calling conventions that already pass arguments on the stack.

The common requirement

Whichever method is used, the kernel must (1) locate the arguments, (2) copy them into kernel memory, and (3) validate every pointer and length before using it.

Pointer Validation: Never Trust User Memory

- A user pointer may point to kernel memory, another process's memory, or unmapped memory.
- The kernel checks the range against the process's address space.
- It copies data with special routines (`copy_from_user`/`copy_to_user` on Linux) that handle faults safely.
- A bad pointer returns `EFAULT`; it does not crash the kernel.

The security rule

Every argument is untrusted input. Validation happens in kernel mode, before the kernel acts on the request.

Socratic Check 2

Question

A user program passes a pointer to a kernel address to `read`. What should happen, and why can the kernel not simply dereference it?

- Dereferencing it would let user code read or overwrite kernel data.
- The kernel checks that the range lies in the user address space.
- If it does not, the call fails with `EFAULT`.

System calls are not a random list. They fall into six categories, and every command you run is a combination of them.

The Six Categories

Category	Purpose	Typical calls
Process control	create, terminate, wait, load	`fork`, `exec`, `wait`, `exit`
File management	create, open, read, write, close	`open`, `read`, `write`, `close`, `lseek`
Device management	request/release, read/write, control	`ioctl`, `read`, `write`
Information maintenance	get/set time, process, file attributes	`getpid`, `alarm`, `sleep`, `stat`
Communication	create/delete connections, send/receive	`pipe`, `mmap`, `socket`, `send`, `recv`
Protection	set/get permissions	`chmod`, `umask`, `chown`, `setuid`

Silberschatz et al., Ch. 2 §2.4, pp. 58–66.

Process Control

- `fork`: create a child process that is a copy of the parent.
- `exec`: replace the current process image with a new program.
- `wait`: block until a child terminates and collect its status.
- `exit`: terminate the calling process and release its resources.

The shell's job

A shell `forks`, the child `execs` the command, and the parent `waits`. `cp` itself is the child after `exec`.

File Management

- `open`: create or open a file, returning a file descriptor.
- `read`/`write`: transfer bytes between user memory and the file.
- `lseek`: move the file offset without transferring data.
- `close`: release the descriptor and flush state.
- `stat`: read metadata (size, owner, permissions, times).

The file descriptor is a small integer index into the process's per-process open-file table — not the file itself.

Device, Information, Communication, Protection

Device management

`ioctl` controls device-specific behavior; `read` and `write` move data. Devices are often accessed through the same descriptor interface as files.

Information maintenance

`getpid`, `alarm`, `sleep`, `time`; `stat` for file metadata.

Communication

`pipe`, `mmap`, `socket`, `send`, `recv`: move data between processes or across a network.

Protection

`chmod`, `umask`, `chown`, `setuid`: set or query access rights.

Worked Trace: cp Across the Categories

1. **Process control:** the shell `forks` and the child `execs` `cp`.
2. **File management:** `open("notes.txt", O_RDONLY)` and `open("backup.txt", O_WRONLY|O_CREAT)`.
3. **File management:** a loop of `read` into a buffer and `write` to the destination.
4. **File management:** `close` both descriptors.
5. **Process control:** `exit(0)`; the shell `waits`.

One shell command, ten or more system calls. The categories are not abstract: they are the steps in the trace.

Socratic Check 3

Question

Which category does `fork` belong to, and why is it not file management even though the child may open files?

- `fork` creates a new process — process control.
- The child inherits the parent's open-file table, but `fork` itself does not create or access a file.
- The category describes the *operation*, not the resources the process may use afterward.

The interface is stable, but every call has a cost. Good systems programmers batch calls and know when the boundary is being crossed.

Numbers, ABI, and Compatibility

- Each architecture defines a system-call number for each service.
- The numbers are part of the **application binary interface**.
- A kernel cannot renumber existing calls without breaking every compiled program.
- New calls are added at the end of the table; old calls remain for compatibility.

Consequence

The system-call table is a long-lived contract between the kernel and every binary that runs on it.

The Cost of Crossing the Boundary

- Save user context and switch to kernel mode.
- Index the table and enter the handler.
- Validate arguments and copy data across the boundary.
- Perform the operation, then restore the user context.

Why batching matters

Copying 1 MB in one `read` costs one mode switch; copying it in 1-byte chunks costs a million. The bytes are the same; the crossings are not.

`strace` shows the actual calls a program makes — useful for seeing the boundary in action.

Common Misconceptions

Wrong

- "`printf` is a system call."
- "A system call is just a function call."
- "The kernel can trust the user's pointer."
- "Every library call traps into the kernel."

Right

- `printf` uses buffered `write` calls.
- It changes privilege level and uses a kernel entry point.
- User pointers and lengths are untrusted and validated.
- Many library calls stay entirely in user space.

Takeaways

- A **system call** is the program's request to the kernel; it is invoked through a trap, not an ordinary call.
- The **system-call number** selects a handler from the kernel's dispatch table.
- Arguments travel in **registers**, a **memory block**, or the **stack**; every pointer and length is validated.
- The six categories — process control, file management, device management, information maintenance, communication, protection — organize the interface.
- Mode switches are expensive; batch work to reduce them.

Next: System Programs and OS Structures

- Today: how a program names a kernel service, how the arguments cross the boundary, and what kinds of calls exist.
- Next lecture: the user-level programs built on top of these calls, and the internal structures — monolithic, layered, microkernel, modular, virtual machine — that organize the kernel itself.

Reading

Silberschatz et al., *Operating System Concepts*, Ch. 2 §2.5–2.8 [**?**]; Stallings, *Operating Systems: Internals and Design Principles*, Ch. 2 and Ch. 4 §4.3 [**?**].

Further Reading (Page-Cited)

- A. Silberschatz, P. B. Galvin, G. Gagne, *Operating System Concepts*, 8e — Ch. 2 §2.3 (system calls, pp. 55–58), §2.4 (types of system calls, pp. 58–66).
- D. P. Bovet & M. Cesati, *Understanding the Linux Kernel*, 3e — Ch. 10 (system calls, pp. 398–419), for the concrete x86/Linux entry and exit path.
- W. Stallings, *Operating Systems: Internals and Design Principles*, 6e — Ch. 3 (process description and control), for the process context in which calls execute.

Page-verified via each book's `index.md` under `master_supporting_docs/CS502/supporting_books/`.

References I


Abraham Silberschatz, Peter B. Galvin, and Greg Gagne.
Operating System Concepts.
John Wiley and Sons, 2018.


William Stallings.
Operating Systems: Internals and Design Principles.
Prentice Hall, 2001.